

ESCOPO DO PROJETO — ESTRUTURA ANALÍTICA (EAP)

ID	Nível	Pacote de Trabalho / Atividade	Descrição	Responsável	Status	Observações
1	1	Protocolo e Parser	Codificação/decodificação e serialização das mensagens do protocolo tracker/cliente e cliente/cliente		Em andamento	
1.1	2	Parser B-encode (encode/decode)	Implementação do parser.py — leitura e escrita de int, byte string, lista e dicionário	Felipe	Concluído	Parser entregue antes da Rodada 1; referência: commits e demonstração.
1.2	2	Testes do parser B-encode	Casos de teste unitários cobrindo encode e decode, incluindo casos de borda		Em andamento	
1.2.1	3	– Testes de decodificação (inteiros, strings, listas, dicionários)	Casos normais de decode para cada tipo bencode	Vinícios	Concluído	Casos normais de decodificação aprovados em testes/test_bencode.py; commit 96dfbf0.
1.2.2	3	– Testes de codificação (dicionário → bencode)	Casos normais de encode a partir de estruturas Python	Vinícios	Concluído	Casos de codificação aprovados em testes/test_bencode.py; commit 96dfbf0.
1.2.3	3	– Testes de casos de borda	Bencode aninhado, valores vazios, strings binárias	Vinícios	Em andamento	Testes de aninhamento e dados binários aprovados; estrutura vazia ainda falha. Issue #21 permanece aberta.
1.3	2	Serialização binária do Peer Wire Protocol (struct)	Empacotamento/desempacotamento das mensagens P2P (length prefix + ID + payload) — ver Anexo A.4		Concluído	
1.3.1	3	– Empacotamento/desempacotamento do Handshake	pstrlen+pstr+reserved+info_hash+peer_id (68 bytes fixos)	Carlos Henrique	Concluído	Serialização do handshake integrada ao módulo de mensagens em 01/10.
1.3.2	3	– Empacotamento/desempacotamento de HAVE e BITFIELD	Mensagens de sinalização de posse de peças	Carlos Henrique	Concluído	Serialização de BITFIELD e HAVE entregue no módulo de mensagens.
1.3.3	3	– Empacotamento/desempacotamento de REQUEST e PIECE	Mensagens de transferência de blocos de 16 KIB	Carlos Henrique	Concluído	Serialização de REQUEST e PIECE entregue no módulo de mensagens.
2	1	Comunicação Cliente–Tracker	Announce request/response e controle de acesso à rede — ver Anexo A.1/A.2		Em andamento	
2.1	2	Cadastro/autorização de clientes na rede	Controle de acesso básico — só clientes autorizados recebem resposta do Tracker (domínio fechado)		Não iniciado	
2.1.1	3	– Definir mecanismo de autorização (lista de peer_id permitidos)			Não iniciado	Replanejado para Sprint 2: segurança/autorização não implementada no MVP.
2.1.2	3	– Implementar verificação de autorização no Tracker			Não iniciado	Replanejado para Sprint 2: Tracker atual ainda não controla clientes autorizados.
2.1.3	3	– Testes de rejeição de cliente não autorizado			Não iniciado	Replanejado para Sprint 2, após implementar autorização.
2.2	2	Implementar Announce Request (cliente → Tracker)	Montagem da requisição HTTP GET com info_hash, peer_id, port, uploaded, downloaded, left e event		Em andamento	
2.2.1	3	– Montagem da query string (info_hash, peer_id, port, compact)		Felipe	Concluído	Query de announce e retorno da resposta corrigidos nos commits 1ebba49, f69db95 e 1cc4ccd.
2.2.2	3	– Cálculo de uploaded/downloaded/left		Luiz Eduardo / Felipe	Em andamento	left e uploaded são atualizados; downloaded ainda não é acumulado durante o download.
2.2.3	3	– Testes do announce request		Vinícios / Luiz Eduardo	Em andamento	Testes C++ de announce e teste manual ponta a ponta aprovados; suite Python do Tracker ainda precisa ser corrigida. Issue #22 aberta.
2.3	2	Implementar Announce Response (Tracker → cliente)	Servidor Tracker mantém info_hash → peers e responde em JSON (interval, peers, complete e incomplete)		Concluído	
2.3.1	3	– Tabela info_hash → lista de peers no Tracker		Luiz Eduardo / Carlos Henrique	Concluído	Persistência por info_hash em SQLite com migração e testes C++ aprovados.
2.3.2	3	– Formato JSON de peers (MVP)	Resposta JSON com peer_id, ip e port; substitui peers compactos/Bencode no MVP.	Luiz Eduardo	Concluído	Resposta JSON adotada no MVP e documentada no README do Tracker.
2.3.3	3	– Contadores complete/incomplete		Luiz Eduardo / Carlos Henrique	Concluído	Contadores complete/incomplete validados no announce do Tracker.
2.4	2	Tratamento da resposta pelo cliente	Cliente interpreta a resposta JSON do Tracker e utiliza a lista de peers		Em andamento	
2.4.1	3	– Parsing da resposta JSON	Cliente interpreta a resposta JSON do Tracker; requer validação ponta a ponta.	Luiz Eduardo / Felipe	Concluído	Resposta JSON interpretada pelo cliente no fluxo completo Tracker–Seeder–Leecher.
2.4.2	3	– Extração e tratamento da lista de peers		Luiz Eduardo / Felipe	Concluído	Lista de peers convertida para (IP, porta) e utilizada pelo download.
2.4.3	3	– Tratamento de failure reason		Luiz Eduardo / Felipe	Em andamento	failure_reason é tratado no cliente, mas falta caso automatizado específico.
3	1	Comunicação entre Clientes (Peer Wire Protocol)	Handshake e troca de fragmentos entre peers — ver Anexo A.3/A.4		Em andamento	
3.1	2	Handshake entre peers	Troca dos 68 bytes fixos e validação do info_hash		Concluído	
3.1.1	3	– Envio do handshake		Felipe / Carlos Henrique	Concluído	Handshake utilizado na transferência e coberto pela suite P2P.

3.1.2	3	– Validação do info_hash recebido		Felipe / Carlos Henrique	Concluído	Validação de info_hash coberta pela suite P2P.
3.2	2	Anúncio de posses (BITFIELD)	Seeder envia vetor de bits indicando as peças que possui (ID 5)		Concluído	
3.2.1	3	– Geração do bitfield local		Felipe / Carlos Henrique	Concluído	BITFIELD gerado pelo seeder; suite P2P aprovada.
3.2.2	3	– Leitura do bitfield remoto		Felipe / Carlos Henrique	Concluído	BITFIELD remoto lido pelo leecher; suite P2P aprovada.
3.3	2	Controle de fluxo (INTERESTED/UNCHOKE)	Leecher sinaliza interesse (ID 2) e Seeder libera banda (ID 1)		Concluído	
3.3.1	3	– Envio de INTERESTED		Felipe / Carlos Henrique	Concluído	INTERESTED utilizado no fluxo ponta a ponta.
3.3.2	3	– Lógica de UNCHOKE (liberação de banda)		Felipe / Carlos Henrique	Concluído	UNCHOKE utilizado no fluxo ponta a ponta.
3.4	2	Loop de requisição e entrega de blocos (REQUEST/PIECE)	Requisição de blocos de 16 KiB (ID 6) e entrega dos dados binários (ID 7)		Concluído	
3.4.1	3	– Envio de REQUEST por bloco (16 KiB)		Felipe	Concluído	REQUEST por bloco executado na transferência completa.
3.4.2	3	– Recebimento e buffer de PIECE		Felipe	Concluído	PIECE recebido e armazenado em buffer durante a transferência.
3.4.3	3	– Controle do loop até completar a peça		Felipe	Concluído	Loop concluiu 260 peças no teste ponta a ponta de 05/10/2026.
3.5	2	Notificação de peça concluída (HAVE)	Nó anuncia aos peers que validou 100% dos blocos de uma peça (ID 4)		Em andamento	
3.5.1	3	– Validação de peça completa (hash)		Felipe	Concluído	SHA-1 por peça validado antes da escrita em disco.
3.5.2	3	– Broadcast de HAVE aos peers			Não iniciado	Mensagem HAVE possui serialização/testes, mas o broadcast não está integrado.
4	1	Download e Persistência	Sessão de download e gravação em disco dos fragmentos recebidos		Em andamento	
4.1	2	Sessão TCP consistente	Conexão estável para receber fragmentos sem perda		Em andamento	
4.1.1	3	– Abertura e manutenção da conexão TCP		Luiz Eduardo / Felipe	Concluído	Conexão TCP manteve a transferência completa Tracker–Seeder–Leecher.
4.1.2	3	– Tratamento de timeout/reconexão		Felipe	Não iniciado	Timeout e reconexão do cliente permanecem no Sprint 2.
4.2	2	Escrita incremental em disco	Gravar peças conforme completam 100% dos blocos no buffer, evitando estourar a RAM		Concluído	
4.2.1	3	– Buffer de peça em memória		Felipe	Concluído	Buffer por peça e reserva do arquivo final implementados.
4.2.2	3	– Escrita da peça validada no arquivo final		Felipe	Concluído	260 peças gravadas incrementalmente no teste ponta a ponta.
4.3	2	Verificação de integridade dos fragmentos	Checagem de hash (SHA-1) por peça recebida antes de gravar/confirmar		Concluído	
4.3.1	3	– Cálculo de hash SHA-1 por peça		Felipe	Concluído	Hash SHA-1 calculado por peça antes da gravação.
4.3.2	3	– Comparação com hash esperado e tratamento de falha		Felipe	Concluído	Arquivo reconstruído com SHA-256 idêntico ao original: 844a330e...e58305.
5	1	Diferenciais (Stretch Goals)	Funcionalidades opcionais, priorizadas conforme tempo disponível — ver Documento de Visão, Seção 9		Em andamento	Risco de escopo — ver aba Riscos
5.1	2	Upload de arquivos (fragmentação e catalogação)	Cliente fragmenta o arquivo e o Tracker cataloga os fragmentos recebidos	Felipe / Luiz Eduardo	Em andamento	Seeder, fragmentação e envio de blocos foram antecipados; catalogação de fragmentos no Tracker não foi concluída.
5.2	2	Distribuição resiliente de fragmentos	Garantir que fragmentos sobrevivam à saída abrupta de 1–2 nós		Não iniciado	
5.3	2	Eleição de um novo Tracker em caso de falha	Mecanismo de escolha de um novo coordenador entre os clientes da rede		Não iniciado	
5.4	2	Modo de lockdown automático da rede	Suspender o compartilhamento de fragmentos se o Tracker for suspeito de comprometimento		Não iniciado	
5.5	2	Gestão de permissões mais granular	Perfis de acesso por cliente, além do controle de acesso básico do pacote 2.1		Não iniciado	
6	1	Testes, Documentação e Entrega	Integração final, documentação e preparação das apresentações		Em andamento	
6.1	2	Testes de integração	Testes fim-a-fim entre Tracker e múltiplos clientes	Equipe de desenvolvimento	Concluído	Teste local em 05/10/2026: Tracker + Seeder + Leecher; download concluído e hash do arquivo conferido.

6.2	2	Documentação (README, licença MIT)	Repositório público, documentado, com licença de código aberto	Lucio / Lucas	Em andamento	<i>README do Tracker e gerência de configuração atualizados; README principal e artefatos finais ainda precisam ser versionados.</i>
6.3	2	Preparação das apresentações	Slides e demonstração para as Rodadas 1, 2 e 3	Lucas / Equipe	Em andamento	<i>Plano e base da apresentação da Rodada 2 em atualização; slides finais ainda pendentes.</i>